

Details of the ML model - LSTM Based Autoencoder:

Fitsync Machine Learning Model Documentation

Overview

The Fitsync ML model is an LSTM-based autoencoder designed for heart rate anomaly detection. It processes heart rate data, normalizes it, extracts sequential patterns, and reconstructs normal heart rate sequences to identify anomalies. The model is trained on sequences of heart rate data and is exported as an ONNX model for deployment.

Data Preprocessing

1. Loading the Data:

- The heart rate data is loaded from a CSV file (`new_heart_rates.csv`).
- The `heart_rates` column contains lists of heart rate values for each user.
- Each row corresponds to a user and contains heart rate time-series data.
- The heart rate values are stored as strings. We convert them into lists of numerical values.

```
import pandas as pd

df = pd.read_csv('new_heart_rates.csv')
df["heart_rates"] = df["heart_rates"].apply(eval)
df.head()
```

1. Normalization:

- Each user's heart rate data is normalized using MinMaxScaler.

- The scaler is stored for future inverse transformation.
- To standardize heart rate values, MinMax Scaling is applied to each user's data.

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

scalers = {}

def normalize_user_data(user_df):
    scaler = MinMaxScaler()
    all_heart_rates = np.concatenate(user_df["heart_rates"].values).reshape(-1,
1)
    normalized_hr = scaler.fit_transform(all_heart_rates).flatten()
    scalers[user_df["user_id"].iloc[0]] = scaler
    split_indices = np.cumsum([len(l) for l in user_df["heart_rates"].values])[:-1]
    normalized_lists = np.split(normalized_hr, split_indices)
    return pd.Series(normalized_lists, index=user_df.index)

df["normalized_heart_rates"] = df.groupby("user_id", group_keys=False).appl
y(normalize_user_data)
df.head()
```

Sequence Preparation

- The model processes data in fixed-length sequences (50 time steps).
- LSTMs process sequential data. We generate sequences of 50 heart rate values.
- Sequences are extracted from normalized heart rate lists for training.

```
SEQ_LENGTH = 50

def create_sequences(hr_list, seq_length):
    sequences = []
```

```

for i in range(len(hr_list) - seq_length + 1):
    sequences.append(hr_list[i : i + seq_length])
return np.array(sequences)

X_train = []
for _, user_df in df.groupby("user_id"):
    user_hr = np.concatenate(user_df["normalized_heart_rates"].values)
    user_sequences = create_sequences(user_hr, SEQ_LENGTH)
    X_train.append(user_sequences)

X_train = np.vstack(X_train)
X_train = np.expand_dims(X_train, axis=-1)
print(f"Final shape of X_train: {X_train.shape}")

```

Model Architecture

The model is an **LSTM Autoencoder**:

- **Encoder:** Two LSTM layers compress the input into a latent representation.
- **Latent Layer:** A fully connected layer reduces dimensionality.
- **Decoder:** Two LSTM layers reconstruct the input from the latent space.
- **Output Layer:** A linear layer generates the final heart rate sequence.

Model Definition

- The Autoencoder consists of an LSTM-based encoder-decoder architecture. The encoder compresses the data into a latent space, and the decoder reconstructs the original sequence.

```

import torch
import torch.nn as nn
import torch.optim as optim

class LSTMAutoencoder(nn.Module):
    def __init__(self, seq_len, n_features, latent_dim=32):

```

```

    super(LSTMAutoencoder, self).__init__()
    self.encoder = nn.LSTM(input_size=n_features, hidden_size=64, num_layers=2, batch_first=True)
    self.latent = nn.Linear(64, latent_dim)
    self.decoder = nn.LSTM(input_size=latent_dim, hidden_size=64, num_layers=2, batch_first=True)
    self.output_layer = nn.Linear(64, n_features)

    def forward(self, x):
        encoded, _ = self.encoder(x)
        latent = self.latent(encoded[:, -1, :])
        latent_repeated = latent.unsqueeze(1).repeat(1, x.shape[1], 1)
        decoded, _ = self.decoder(latent_repeated)
        reconstructed = self.output_layer(decoded)
        return reconstructed

seq_len = 50
n_features = 1
model = LSTMAutoencoder(seq_len, n_features)

```

Training Setup

- **Loss Function:** Mean Squared Error (MSE) loss to minimize reconstruction error.
- **Optimizer:** Adam optimizer with learning rate 0.001.
- **Training Loop:** Trains the model for 5 epochs.
- The data is converted into a PyTorch DataLoader for efficient training.
- We use Mean Squared Error (MSE) as the loss function and Adam optimizer.

```

from torch.utils.data import DataLoader, TensorDataset

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
train_dataset = TensorDataset(X_train_tensor)

```

```

train_loader = DataLoader(train_dataset, batch_size=256, shuffle=True)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
model.to(device)

num_epochs = 5
for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0
    for batch in train_loader:
        batch = batch[0].to(device)
        optimizer.zero_grad()
        reconstructed = model(batch)
        loss = criterion(reconstructed, batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss / len(train_loader):.6f}")

```

Model Saving & Export

- The trained model is saved as a PyTorch `.pth` file.
- The model is also exported to the ONNX format for deployment.
- To integrate the model with other platforms like MATLAB or edge devices, we export it to ONNX.

```

torch.save(model.state_dict(), "heart_rate_predictor_lstm_model.pth")

import torch.onnx
dummy_input = torch.randn(1, 50, 1).to(device)
torch.onnx.export(model, dummy_input, "lstm_model.onnx",

```

```
export_params=True, input_names=["input"], output_names=["output"])
```

Conclusion

This LSTM-based autoencoder learns normal heart rate patterns and can reconstruct normal sequences. The reconstruction error can be used for **anomaly detection**, identifying significant deviations from expected heart rate behavior. The ONNX model allows easy deployment in real-time monitoring systems like Fitsync.

Key Features

- **Normalization:** Scales heart rate data between 0 and 1.
- **Sequence Generation:** Transforms heart rate readings into time-series sequences.
- **LSTM Autoencoder:** Learns normal heart rate patterns and reconstructs sequences.
- **Anomaly Detection:** High reconstruction error indicates anomalies.
- **ONNX Export:** Enables deployment in MATLAB and embedded systems.

Possible VIVA Questions based on the ML Model